STM32CubeMX 使用DMA 实现不定长数据收发

这里教程是选择的STM32F407。STM32F103也测试过。

1.首先选择串口，选择异步通信。

2.添加DMA

3.打开中断

4.生成代码，生成代码选择了每个外设单独使用.c/.h

5.我使用的是Keil5。打开工程，首先注释掉dma.c里的DMA接收中断，因为不需要DMA接收中断，DMA发送中断是需要的。(dma.c)

```
void MX_DMA_Init(void)
{
  /* DMA controller clock enable */
  __HAL_RCC_DMA2_CLK_ENABLE();

  /* DMA interrupt init */
  /* DMA2_Stream0_IRQn interrupt configuration */
  HAL_NVIC_SetPriority(DMA2_Stream0_IRQn, 2, 0);
  HAL_NVIC_EnableIRQ(DMA2_Stream0_IRQn);
  /* DMA2_Stream2_IRQn interrupt configuration */
//  HAL_NVIC_SetPriority(DMA2_Stream2_IRQn, 0, 0);
//  HAL_NVIC_EnableIRQ(DMA2_Stream2_IRQn);
  /* DMA2_Stream7_IRQn interrupt configuration */
  HAL_NVIC_SetPriority(DMA2_Stream7_IRQn, 1, 0);
  HAL_NVIC_EnableIRQ(DMA2_Stream7_IRQn);

}
```

6.在usart.h文件里定义串口接收数据类型我是如下定义的(usart.h)

```
/* USER CODE BEGIN Private defines */
#define RECEIVELEN 1024
#define USART_DMA_SENDING 1//发送未完成
#define USART_DMA_SENDOVER 0//发送完成
typedef struct
{
uint8_t receive_flag:1;//空闲接收标记
uint8_t dmaSend_flag:1;//发送完成标记
uint16_t rx_len;//接收长度
uint8_t usartDMA_rxBuf[RECEIVELEN];//DMA接收缓存
}USART_RECEIVETYPE;

extern USART_RECEIVETYPE UsartType1;


/* USER CODE END Private defines */
```

7.然后在usart.c里需要添加空闲中断处理函数，以及DMA发送函数等。(usart.c)

首先定义串口数据类型：

```
/* USER CODE BEGIN 0 */

USART_RECEIVETYPE UsartType1;

/* USER CODE END 0 */
```

然后定义空闲中断处理函数和DMA发送函数

```
/* USER CODE BEGIN 1 */
#ifdef __GNUC__

```

```
  /* With GCC/RAISONANCE, small printf (option LD Linker->Libraries->Small printf
set to 'Yes') calls __io_putchar() */
#define PUTCHAR_PROTOTYPE int __io_putchar(int ch)
#else

  #define PUTCHAR_PROTOTYPE int fputc(int ch, FILE *f)
#endif /* __GNUC__ */

PUTCHAR_PROTOTYPE
{
    HAL_UART_Transmit(&huart1 , (uint8_t *)&ch, 1, 0xFFFF);
    return ch;
}

//DMA发送函数
void Usart1SendData_DMA(uint8_t *pdata, uint16_t Length)
{
    while(UsartType1.dmaSend_flag == USART_DMA_SENDING);
    UsartType1.dmaSend_flag = USART_DMA_SENDING;
    HAL_UART_Transmit_DMA(&huart1, pdata, Length);
}

//DMA发送完成中断回调函数
void HAL_UART_TxCpltCallback(UART_HandleTypeDef *huart)
{
     __HAL_DMA_DISABLE(huart->hdmatx);
    UsartType1.dmaSend_flag = USART_DMA_SENDOVER;
}

//串口接收空闲中断
void UsartReceive_IDLE(UART_HandleTypeDef *huart)
{
    uint32_t temp;

    if((__HAL_UART_GET_FLAG(huart,UART_FLAG_IDLE) != RESET))
    {
        __HAL_UART_CLEAR_IDLEFLAG(&huart1);
        HAL_UART_DMAStop(&huart1);
        temp = huart1.hdmarx->Instance->NDTR;
        UsartType1.rx_len =  RECEIVELEN - temp;
        UsartType1.receive_flag=1;
        HAL_UART_Receive_DMA(&huart1,UsartType1.usartDMA_rxBuf,RECEIVELEN);
    }
}

/* USER CODE END 1 */
```

8.在中断文件里添加(当然，上面的空闲中断处理函数需要**)(stm32f4xxit.c)

9.主函数里，打开空闲中断，初始化DMA接收

```
/* USER CODE BEGIN 2 */
    HAL_UART_Receive_DMA(&huart1, UsartType1.usartDMA_rxBuf, RECEIVELEN);
    __HAL_UART_ENABLE_IT(&huart1, UART_IT_IDLE);
  /* USER CODE END 2 */
```

10.然后就可以在while(1)里处理数据了

```
/* Infinite loop */
  /* USER CODE BEGIN WHILE */

  while (1)
  {
      if(UsartType1.receive_flag)//如果产生了空闲中断
      {
          UsartType1.receive_flag=0;//清零标记
          Usart1SendData_DMA(UsartType1.usartDMA_rxBuf,UsartType1.rx_len);//串口打印收到的数据。
```

```
    }
/* USER CODE END WHILE */

/* USER CODE BEGIN 3 */

}
/* USER CODE END 3 */
```